

Análisis de Algoritmos I

Recursión / Recursividad

L.C.C. María Janneth Rivera Reyna

Lic. En Ciencias de la Computación
Departamento de Matemáticas
Universidad de Sonora

Ciclo 2026-1

Recursión / Recursividad

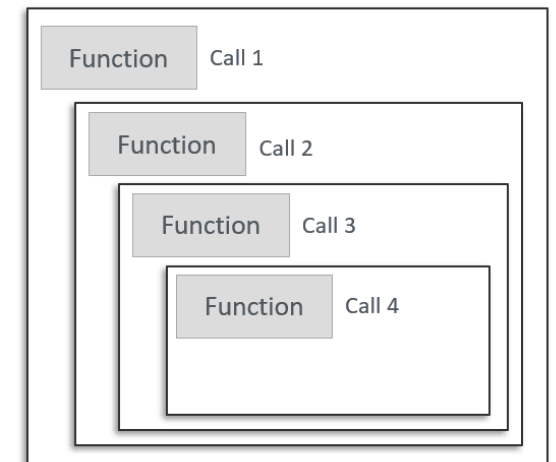
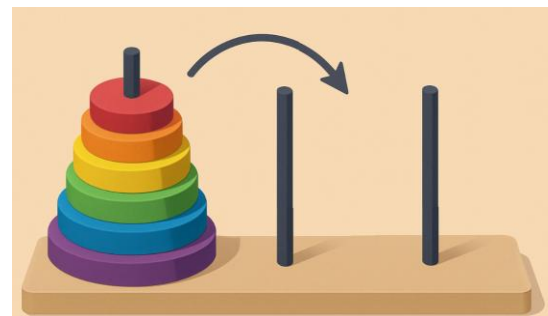
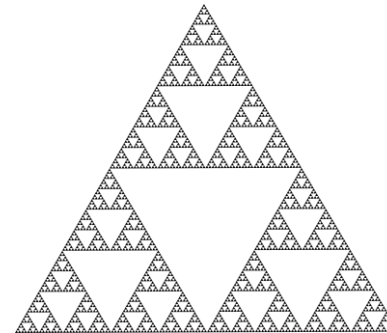


Recursión: repaso

- Técnica que nos permite diseñar un algoritmo para resolver un problema de tal modo que resuelva una instancia más pequeña del mismo problema, hasta que el problema sea tan pequeño que lo podamos resolver directamente. A esta técnica la llamamos **recursión** o **recursividad**.

Aplicaciones:

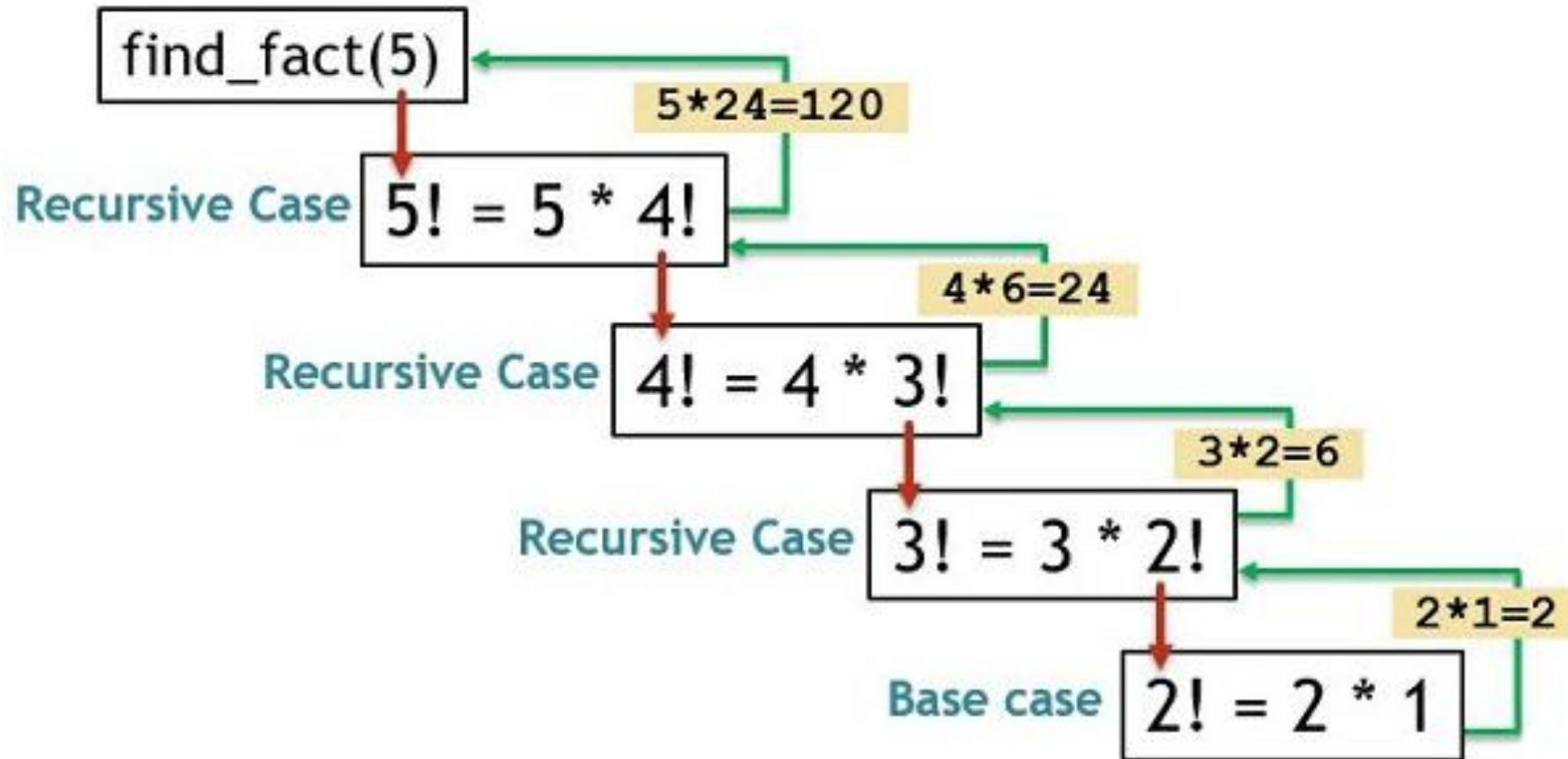
- calcular la función factorial
- determinar si una palabra es un palíndromo
- calcular potencias de un número
- dibujar un tipo de fractal
- problema de las Torres de Hanoi



Recursión: repaso

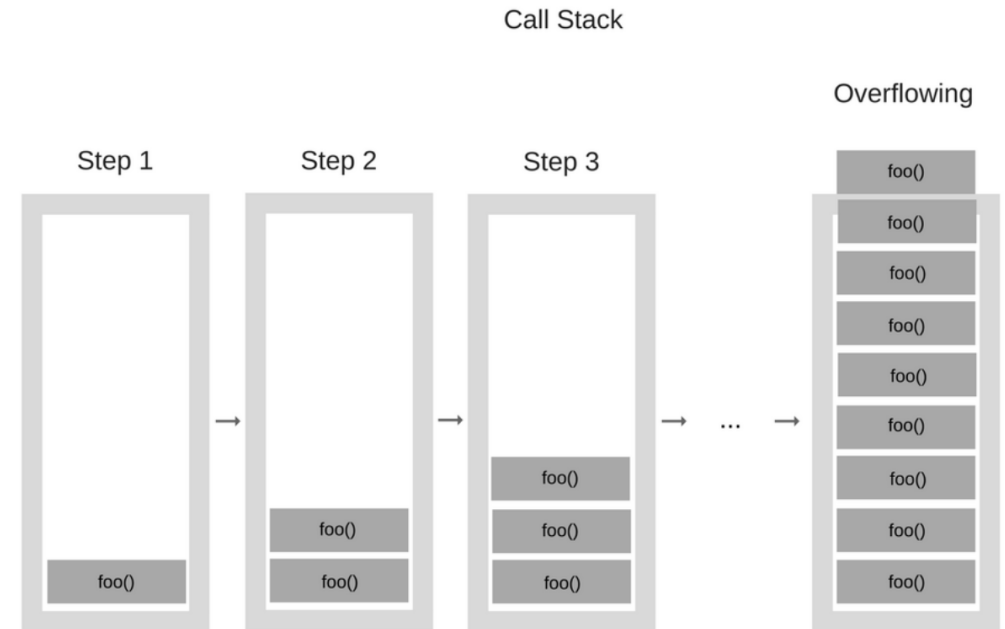
- Recursión es cuando una función se llama a sí misma.
- Una función recursiva tiene dos propiedades:
 - ✓ Una relación de recurrencia
 - ✓ Una condición de paro (caso base)
- La *relación de recurrencia* define **cómo el problema original se reduce** a una o más versiones más pequeñas de sí mismo.
- La *condición de paro* o **caso base** representa **la instancia más pequeña** posible del problema, la que ya no puede dividirse más. Será la que detenga las llamadas recursivas para evitar un **bucle infinito**.

Recursión: Ejemplo Factorial



Recursión: repaso

- Una función recursiva utiliza una **pila de llamadas (call stack)**.
- Almacena la información sobre las subrutinas activas de un programa de computadora en una estructura dinámica de datos **LIFO** (una pila).
- Cuando se consume todo el espacio asignado a la pila, ocurre el error llamado **Stack Overflow** (desbordamiento de pila).



Análisis de un algoritmo recursivo

Cuando un algoritmo es recursivo, no podemos saber directamente cuánto tardará porque el tiempo de un problema grande depende del tiempo de los problemas más chicos:

¿Cuántas llamadas recursivas hay? ¿Cuánto se divide el problema? ¿Qué operaciones adicionales hay?

Cuando un algoritmo contiene una llamada recursiva a sí mismo, a menudo podemos describir su tiempo de ejecución mediante una **ecuación de recurrencia** $T(n)$, que describe el tiempo de ejecución total en un problema de tamaño n en términos del tiempo de ejecución en entradas más pequeñas.

Por ejemplo:

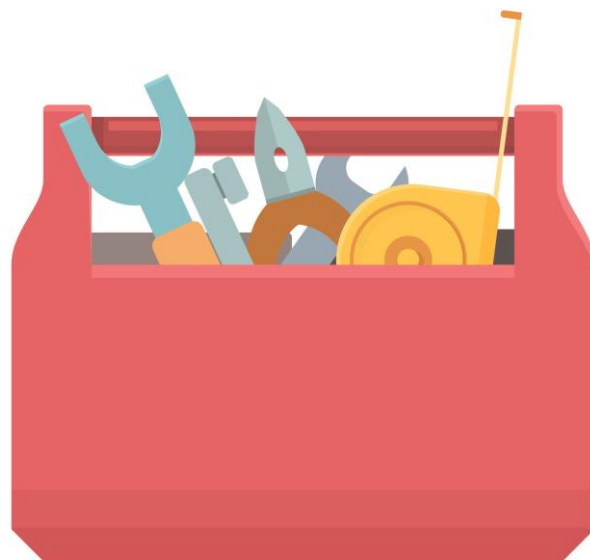
$$T(n) = T\left(\frac{n}{2}\right) + c$$

Métodos para resolver una ecuación de recurrencia $T(n)$

Dada una ecuación de recurrencia **$T(n)$** , para poder simplificarla con $O(T(n))$, necesitamos “romper” la recursión y convertirla en una función simple.

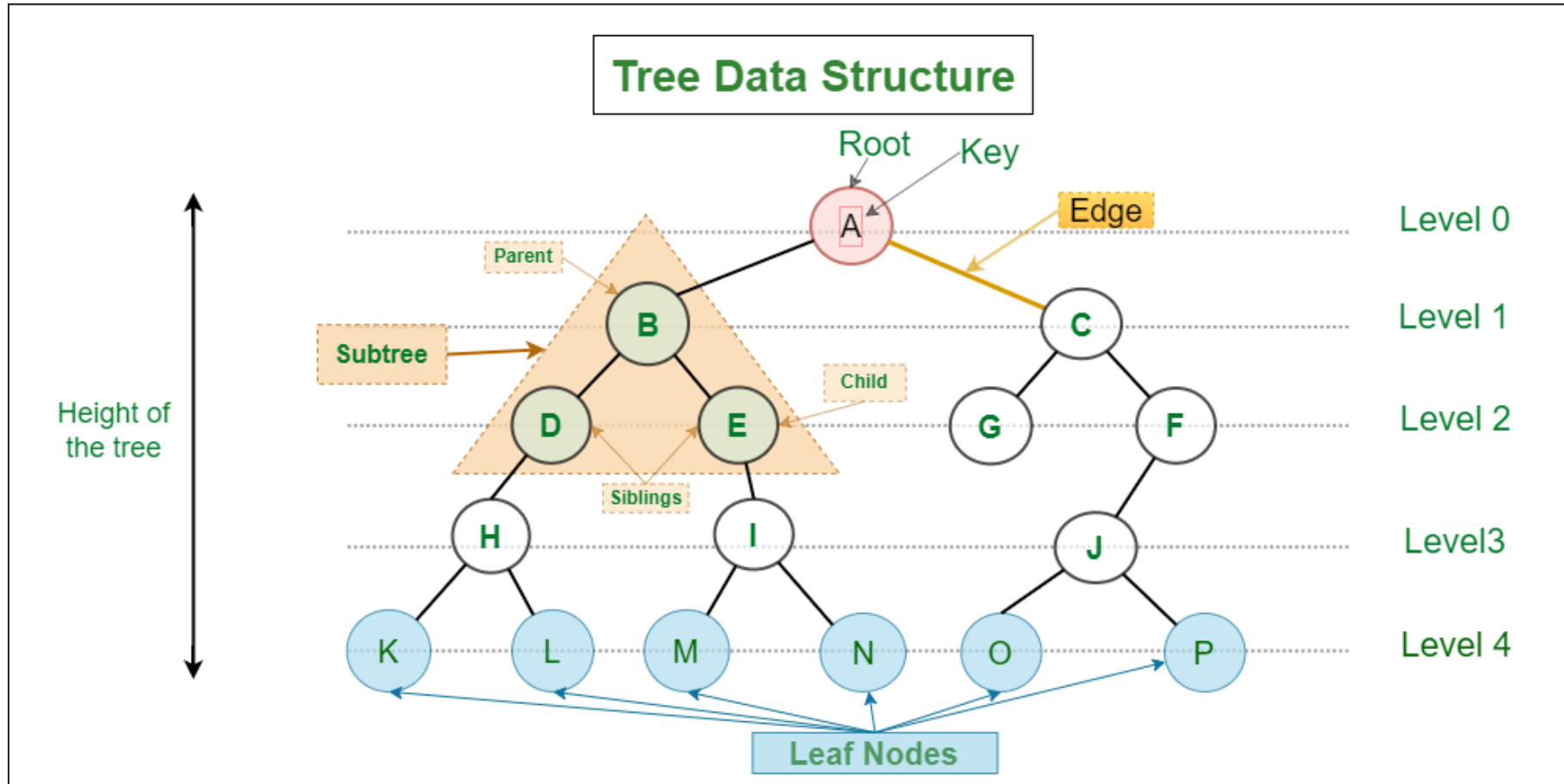
¿Cómo se resuelve una ecuación de recurrencia?

Método	¿Cuándo usarlo?	Ventaja principal	Desventaja
Sustitución (Inducción)	Cuando ya sospechas la respuesta y quieres demostrarla formalmente .	Es el único 100% riguroso matemáticamente.	Requiere buenas bases de álgebra e inducción matemática.
 Árbol de Recursión	Cuando quieres visualizar cómo se divide el trabajo.	Es el más intuitivo .	Puede volverse tedioso con ecuaciones muy complejas.
 Teorema Maestro	Para ecuaciones de la forma $T(n) = aT(n/b) + f(n)$.	Es rápido . Es una "receta" que aplicas y listo.	Solo sirve si el problema se divide en partes iguales. No sirve para $T(n) = T(n-1) + n$.



Árbol de recursión

Árbol de recursión



Árbol de recursión

- Es una **representación visual de las llamadas recursivas** realizadas durante la ejecución de un algoritmo recursivo. Nos sirve para ver qué sucede al iterar una recursión.
- En un árbol de recursión:
 - El **nodo raíz** representa el problema original de tamaño n .
 - Los **hijos de un nodo** representan los subproblemas creados por las llamadas recursivas realizadas por esa función.
 - Los **nodos hoja** representan casos base o llamadas no recursivas.

Árbol de recursión

¿Cómo calcular la complejidad de un algoritmo recursivo?

1. **Dibujar el árbol recursivo** para la relación de recurrencia dada.
2. **Calcular el costo en cada nivel** y contar el **número total de niveles** en el árbol de recursión.
3. **Sumar el costo total** de todos los niveles en el árbol recursivo.
 - a. Si el costo es igual para todos los niveles:

Costo total = costo por nivel * número de niveles

Árbol de recursión

- b. Si el costo cambia en cada nivel (crece o decrece), usamos la **Serie Geométrica**:

$$\text{Costo total} = \text{costo inicial} * \left(\frac{r^k - 1}{r - 1} \right)$$

Donde:

- **r (la razón)**: ¿Por cuánto se multiplica el costo de un nivel a otro?
- **k**: El número total de niveles

Nivel	Costo por nivel
0	n
1	2n
2	4n
3	8n

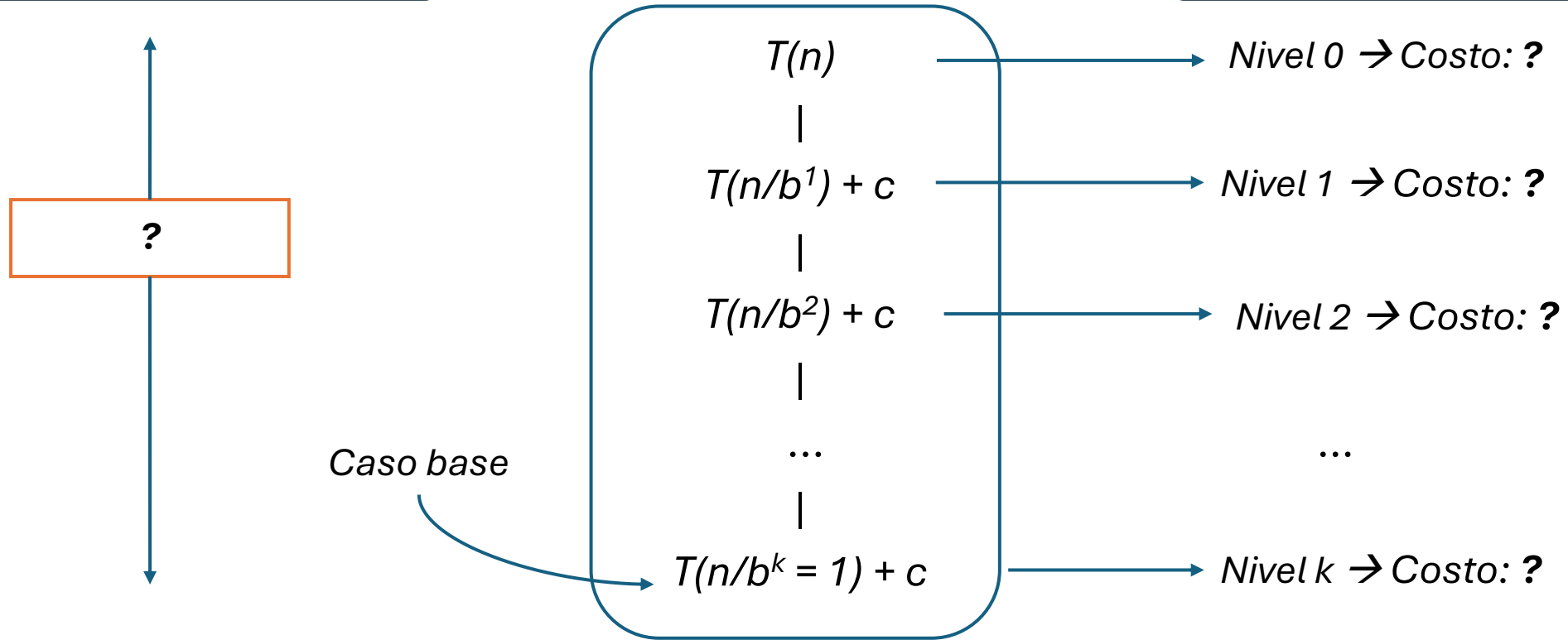
R (razón) = 2 (porque $n * 2 = 2n$ y $2n * 2 = 4n$, y así sucesivamente).

Árbol de recursión: Ejemplo genérico

$$T(n) = T\left(\frac{n}{b}\right) + c$$

¿Cuántos k niveles hay?

¿Cuánto cuesta cada nivel?



$$\begin{array}{l} \text{Costo total} = \text{Costo por nivel} * \text{número de niveles} \\ = \quad ? \quad * \quad ? \end{array}$$

Árbol de recursión: ¿Cuánto cuesta cada nivel?

Para calcular el costo total de un nivel k , se deben multiplicar dos factores:

1. Número de nodos en el nivel k

Si en cada paso el problema se divide en a subproblemas, en el nivel k habrá:

$$\text{Núm. de nodos} = a^k$$

Nivel	Num. nodos
0	1
1	a^1
2	a^2
...	...
k	a^k

2. Costo de cada nodo individual

Si el tamaño original n se divide por b en cada nivel, el tamaño del problema en el nivel k es n/b^k . Por lo tanto, el trabajo realizado por un solo nodo es:

$$\text{Costo de un nodo} = f(n/b^k)$$

$$\text{Costo del nivel } k = (\text{Número de nodos}) * (\text{Costo del nodo})$$

$$C_i = a^k * f(n/b^k)$$

Árbol de recursión: ¿Cuántos niveles hay?

¿Cuándo termina la recursión?

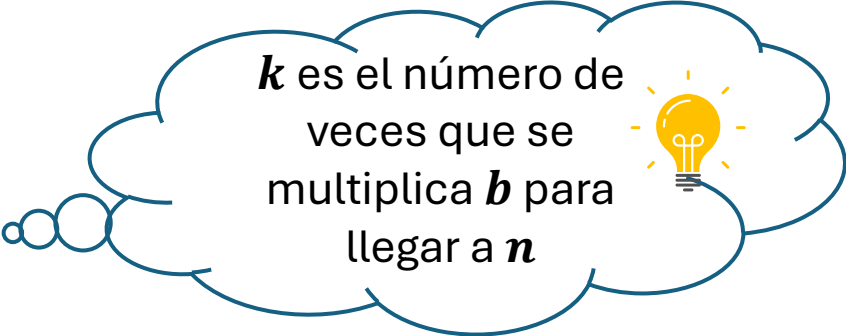
La recursión termina cuando el tamaño llega a 1:

$$\frac{n}{b^k} = 1$$

Ese ***k*** será el número de niveles.

Despejando ***k*** :

$$n = b^k$$



k es el número de veces que se multiplica ***b*** para llegar a ***n***

Aplicando **logaritmos**:

$$k = \log_b n$$

$$\text{Número de niveles (k)} = \log_b n + 1$$

El árbol tiene **$\log_b n + 1$** niveles (contando el nivel del nodo raíz)

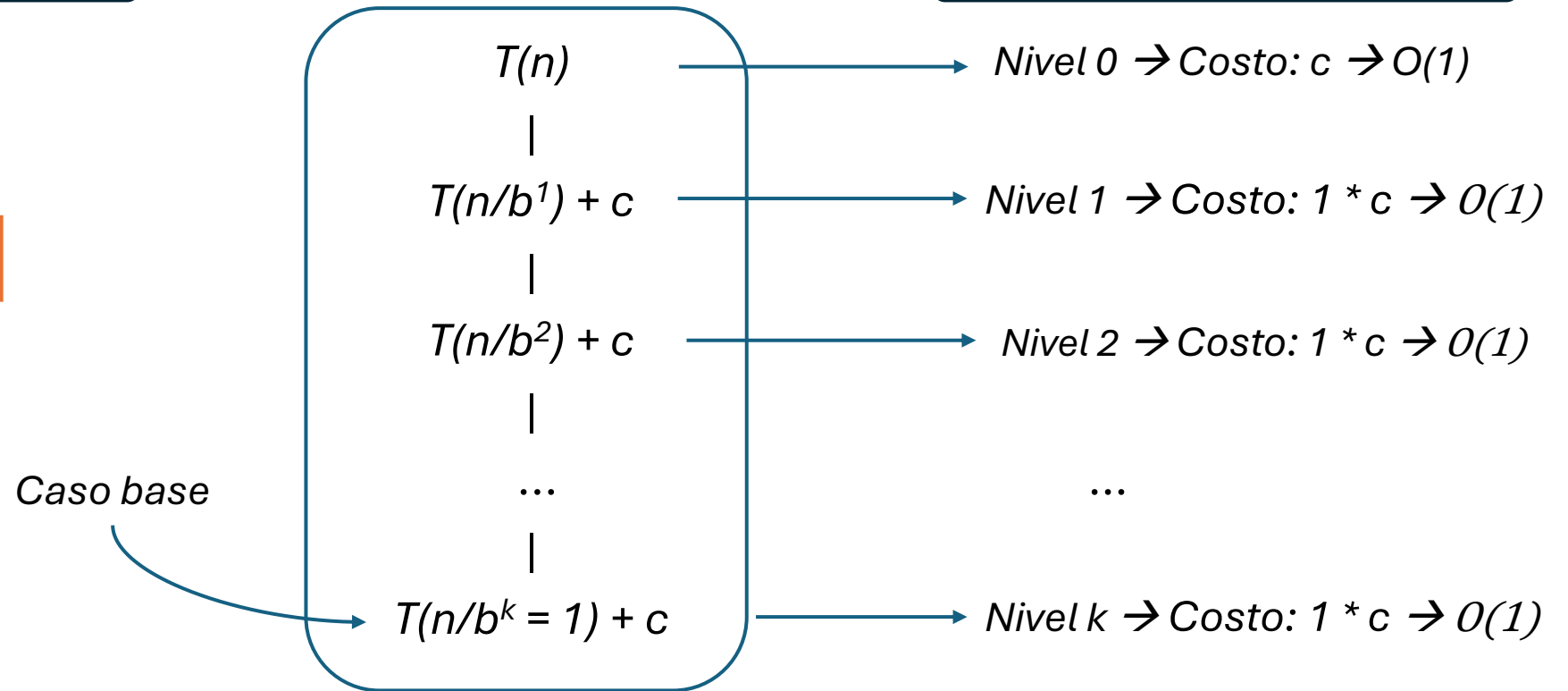
Árbol de recursión: Ejemplo genérico

$$T(n) = T\left(\frac{n}{b}\right) + c$$

¿Cuántos k niveles hay?

$$k = \log_b n + 1$$

¿Cuánto cuesta cada nivel?



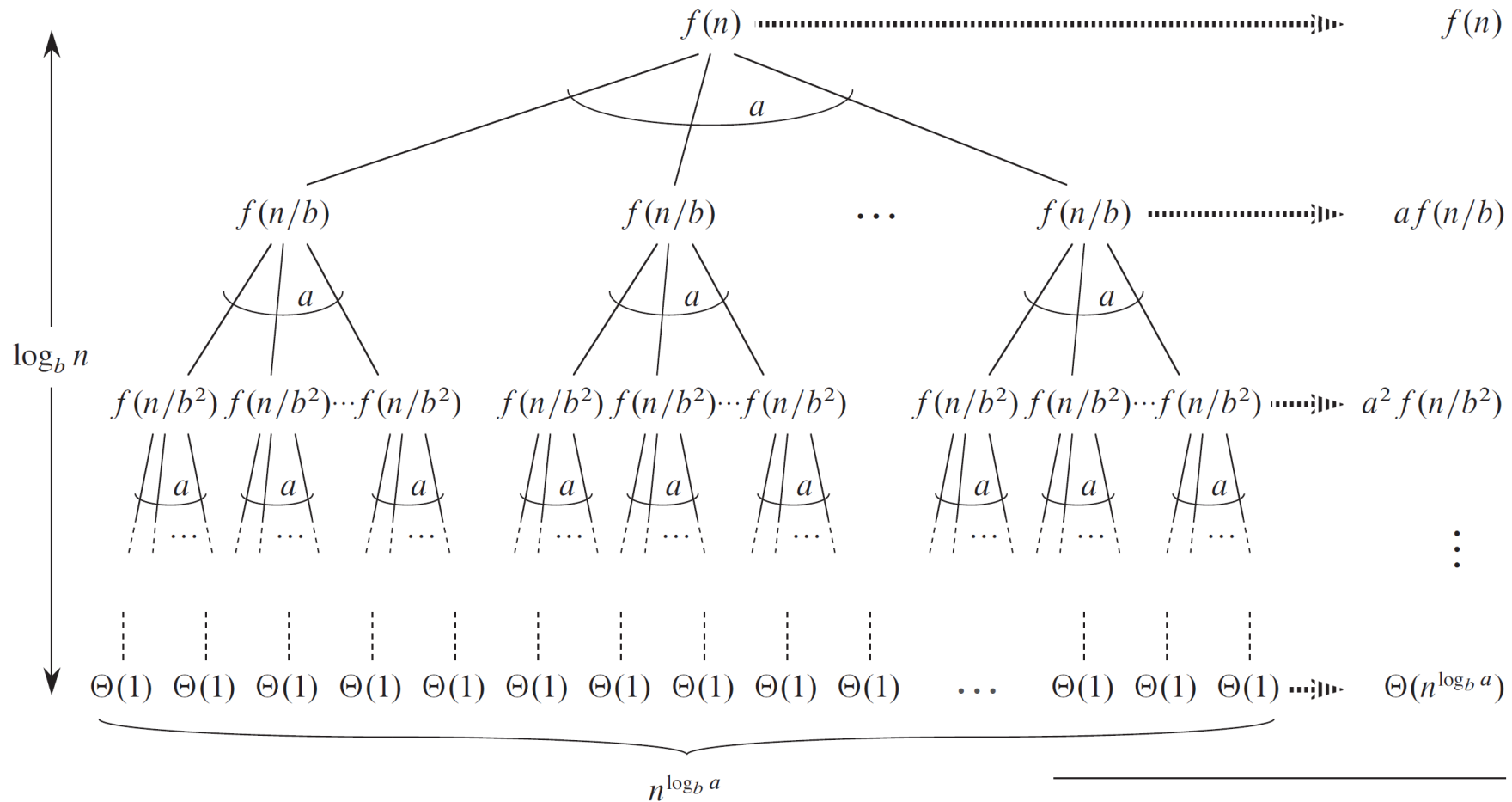
Costo total = Costo por nivel * número de niveles

$$= (1 * c) * \log_b n + 1$$

$$= c * \log_b n + 1$$

$$O(\log_b n)$$

Árbol de recursión

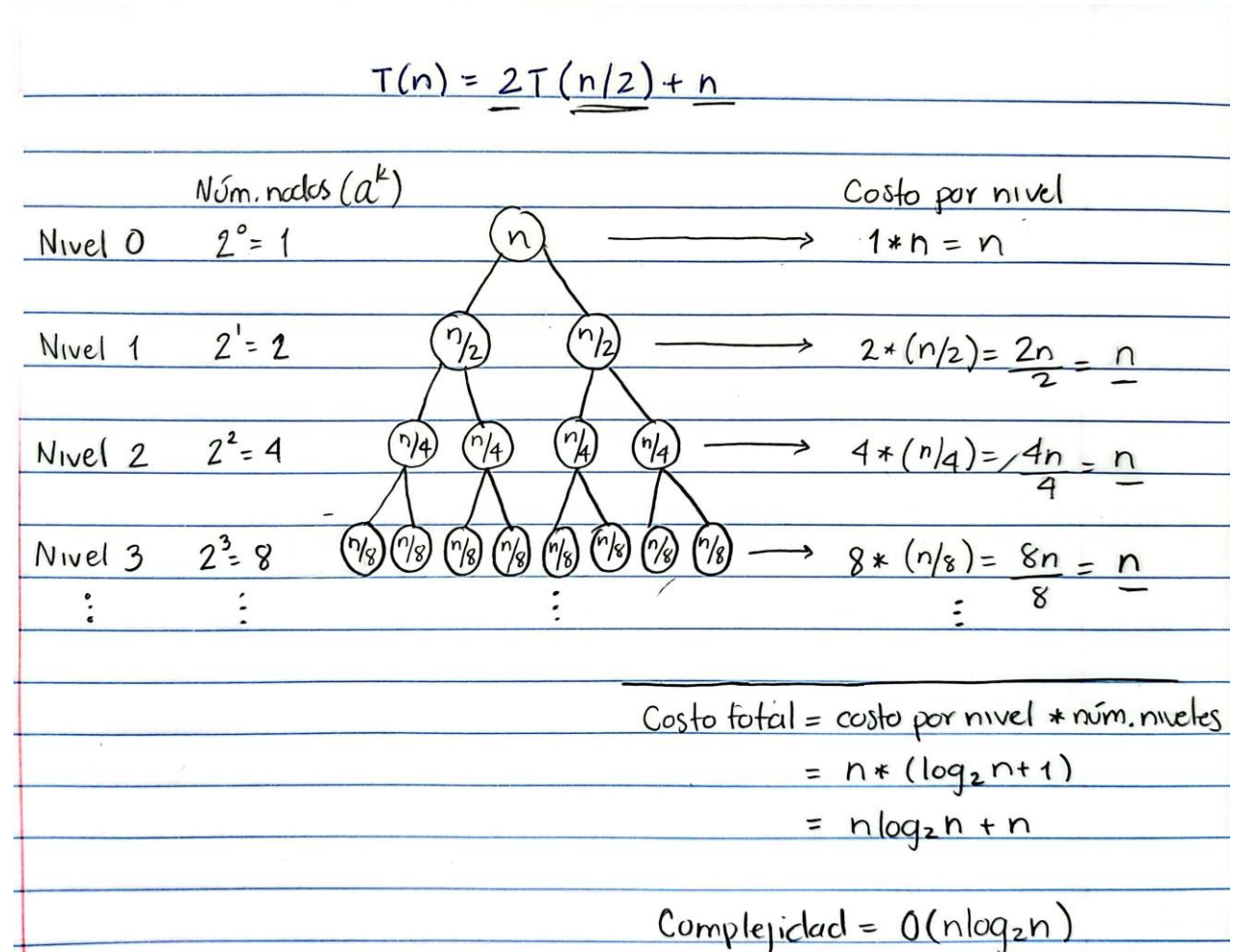


$$\text{Total: } \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j)$$

Árbol de recursión: Ejemplo 1

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

- **Nivel 0 (Raíz):**
 - 1 nodo de costo n
- **Nivel 1:**
 - 2 nodos de tamaño $n/2$.
 - Costo por nodo es $n/2$.
 - El costo total del nivel es $2 * (n/2) = n$.
- **Nivel 2:**
 - 4 subproblemas de tamaño $n/4$.
 - Costo por nodo es $n/4$
 - El costo total del nivel es $4 * (n/4) = n$
- **Nivel 3:**
 - 8 subproblemas de tamaño $n/8$
 - Costo por nodo es $n/8$
 - El costo total del nivel es $8 * (n/8) = n$



Árbol de recursión: Ejemplo 1

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Nivel	Número de nodos	Tamaño del subproblema	Costo por nodo	Costo total por nivel
0	$2^0 = 1$	n	n	n
1	$2^1 = 2$	$n/2$	$n/2$	$2 * (n/2) = n$
2	$2^2 = 4$	$n/4$	$n/4$	$4 * (n/4) = n$
3	$2^3 = 8$	$n/8$	$n/8$	$8 * (n/8) = n$
...	...	...	...	...
k	2^k	$n/2^k$	$n/2^k$	$2^k * \left(\frac{n}{2^k}\right) = n$

Costo por nivel = n

Núm. Niveles = $\log_2 n + 1$

Costo total = costo por nivel * núm. Niveles

$$= n * (\log_2 n + 1)$$

$$= n \log_2 n + n$$

Complejidad: $O(n \log n)$

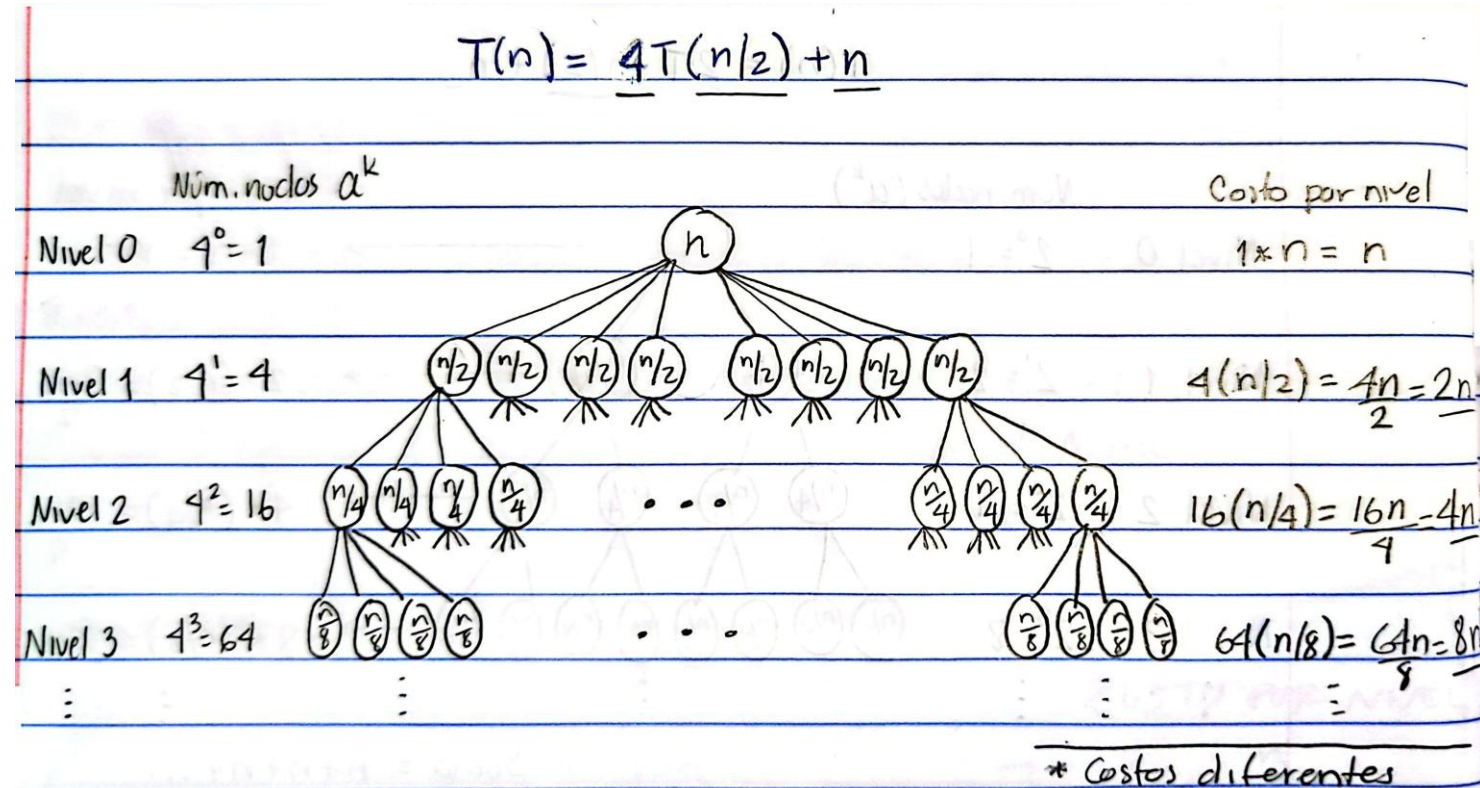
El costo es **igual** en cada nivel



Árbol de recursión: Ejemplo 2

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

- **Nivel 0 (Raíz):**
 - 1 nodo de costo n
- **Nivel 1:**
 - 4 subproblemas (nodos) de tamaño $n/2$.
 - Costo por nodo es $n/2$.
 - El costo total del nivel es $4 * (n/2) = 2n$.
- **Nivel 2:**
 - 16 subproblemas de tamaño $n/4$.
 - Costo por nodo es $n/4$
 - El costo total del nivel es $16 * (n/4) = 4n$
- **Nivel 3:**
 - 64 subproblemas de tamaño $n/8$
 - Costo por nodo es $n/8$
 - El costo total del nivel es $64 * (n/8) = 8n$



Árbol de recursión: Ejemplo 2

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

Nivel	Número de nodos	Tamaño del subproblema	Costo por nodo	Costo total por nivel
0	$4^0 = 1$	n	n	n
1	$4^1 = 4$	n/2	n/2	$4 * (n/2) = 2n$
2	$4^2 = 16$	n/4	n/4	$16 * (n/4) = 4n$
3	$4^3 = 64$	n/8	n/8	$64 * (n/8) = 8n$
...	...	...	...	...
k	4^k	$n/2^k$	$n/2^k$	$4^k * \left(\frac{n}{2^k}\right) = 2^k n$

El costo es **diferente** en cada nivel

Aplicando la serie geométrica:

$$n * (1 + 2 + 4 + 8 + \dots + 2^{\log_2 n})$$

$$\text{Costo total} = n * \left(\frac{r^k - 1}{r - 1}\right), r = 2$$

$$= n * \left(\frac{2^{\log_2 n + 1} - 1}{2 - 1}\right)$$

$$= n * ((2^{\log_2 n} * 2^1) - 1)$$

$$= n * ((n * 2) - 1)$$

$$= n * (2n - 1)$$

Propiedades de las potencias y logaritmos

$$\text{Costo total} = 2n^2 - n$$

$$\text{Complejidad: } O(n^2)$$

Árbol de recursión: Ejemplo 3

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

- **Nivel 0 (Raíz):**

- 1 nodo de costo n^2

- **Nivel 1:**

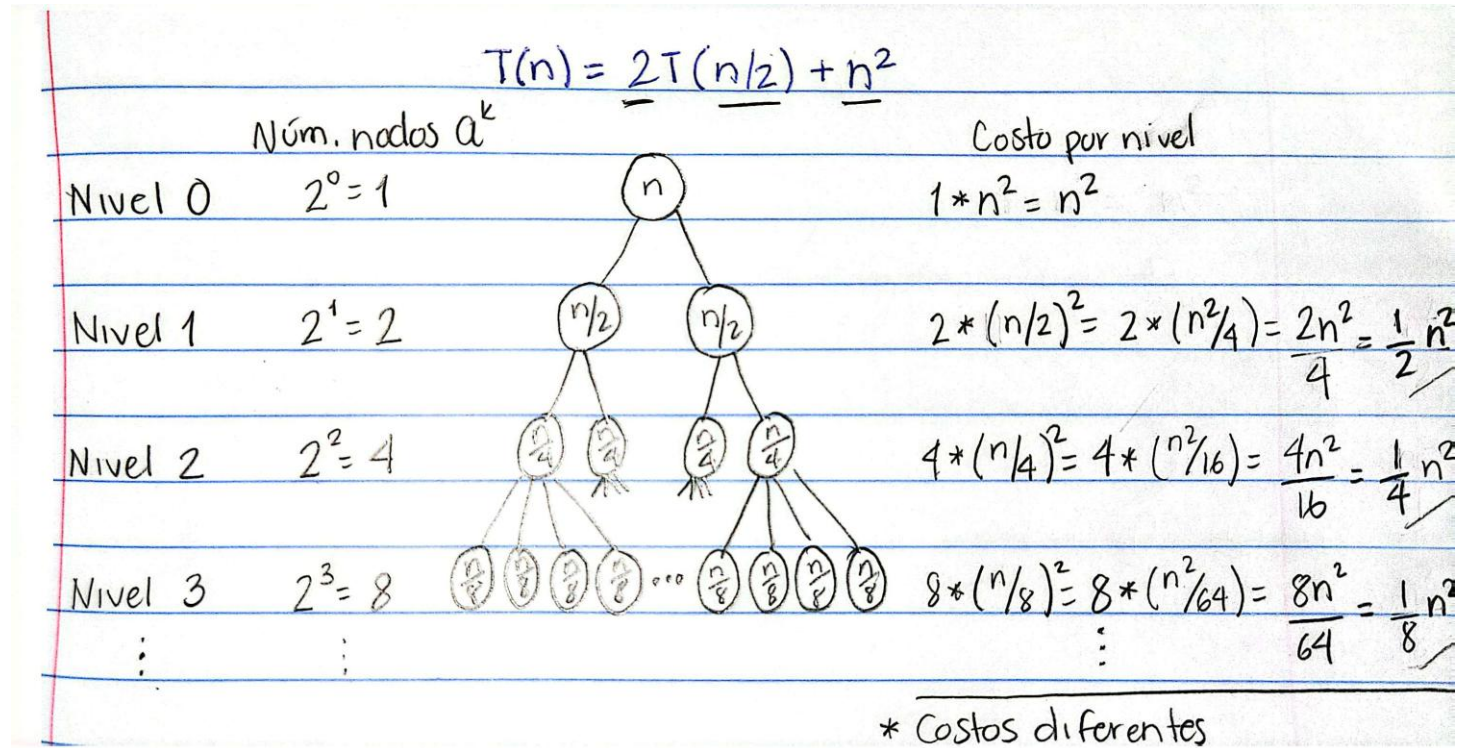
- 2 nodos de tamaño $n/2$.
- Costo por nodo es $(n/2)^2 = n^2/4$
- El costo total del nivel es $2 * (n^2/4) = n^2/2$.

- **Nivel 2:**

- 4 subproblemas de tamaño $n/4$.
- Costo por nodo es $(n/4)^2 = n^2/16$
- El costo total del nivel es $4 * (n^2/16) = n^2/4$

- **Nivel 3:**

- 8 subproblemas de tamaño $n/8$
- Costo por nodo es $(n/8)^2 = n^2/64$
- El costo total del nivel es $8 * (n^2/64) = n^2/8$



Árbol de recursión: Ejemplo 3

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

Nivel	Número de nodos	Tamaño del subproblema	Costo por nodo	Costo total por nivel
0	$2^0 = 1$	n	n^2	$1 * n^2 = n^2$
1	$2^1 = 2$	$n/2$	$(n/2)^2 = n^2/4$	$2 * (n^2/4) = 1/2 n^2$
2	$2^2 = 4$	$n/4$	$(n/4)^2 = n^2/16$	$4 * (n^2/16) = 1/4 n^2$
3	$2^3 = 8$	$n/8$	$(n/8)^2 = n^2/64$	$8 * (n^2/64) = 1/8 n^2$
...	...	...	...	...
k	2^k	$n/2^k$	$(n/2^k)^2$	$2^k * (n/2^k)^2 = n^2(1/2)^k$

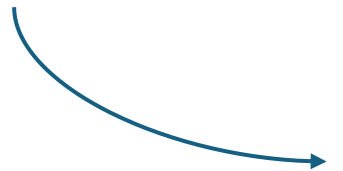
El costo es **diferente** en cada nivel

Aplicando la serie geométrica:

$$n^2 \left(1 + \frac{1}{2} + \frac{1}{4} + \dots + \frac{1}{2^{\log_2 n + 1}} \right)$$

$$\text{Costo total} = n^2 * \left(\frac{r^k - 1}{r - 1} \right), r = \frac{1}{2}$$

$$= n^2 * \left(\frac{(1/2)^{\log_2 n + 1} - 1}{(1/2) - 1} \right)$$



Árbol de recursión: Ejemplo 3 (resolviendo la ecuación de costo total)

Aplicando la serie geométrica:

$$n^2 \left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots + \frac{1}{2^{\log_2 n}} \right)$$

$$\text{Costo total} = n^2 * \left(\frac{r^k - 1}{r - 1} \right), r = \frac{1}{2}$$

$$= n^2 * \left(\frac{(1/2)^{\log_2 n + 1} - 1}{(1/2) - 1} \right)$$

$$\text{Costo total } T(n) = 2n^2 - n$$

Complejidad: $O(n^2)$

A) Resolvemos la ecuación:

Serie geométrica:

$$n^2 * \left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots + \left(\frac{1}{2} \right)^{\log_2 n} \right)$$

$$\text{Costo total p/ último nivel} = 2^k * (n/2^k)^2$$

$$\text{Costo total} = n^2 * \left(\frac{r^k - 1}{r - 1} \right), r = \frac{1}{2}$$

$$= 2^k * \frac{n^2}{2^{2k}} = \frac{2^k n^2}{2^{2k}} = \frac{n^2}{2^k}$$

$$= n^2 * \left(\frac{(1/2)^{\log_2 n + 1} - 1}{(1/2) - 1} \right) \leftarrow a^{k+1} = a^k \cdot a$$

$$= n^2 \left(\frac{1}{2} \right)^k = n^2 \left(\frac{1}{2} \right)^{\log_2 n}$$

$$= n^2 * \left(\frac{(1/2)^{\log_2 n} * (1/2)^1 - 1}{-(1/2)} \right) \leftarrow \left(\frac{a}{b} \right)^{\log_b a} = 1$$

$$= n^2 * \left(\frac{\left(\frac{1}{2^{\log_2 n}} \right) * \left(\frac{1}{2} \right) - 1}{-(1/2)} \right) \leftarrow 1^x = 1$$

$$= n^2 * \left(\frac{\left(\frac{1}{2^{\log_2 n}} \right) * \left(\frac{1}{2} \right) - 1}{-(1/2)} \right) \leftarrow b \log^{b(x)} = x$$

$$= n^2 * \left(\frac{\left(\frac{1}{n} \right) * \left(\frac{1}{2} \right) - 1}{-(1/2)} \right)$$

$$= n^2 * \left(\frac{\left(\frac{1}{n} \right) * \left(\frac{1}{2} \right) - 1}{-(1/2)} \right)$$

$$= n^2 * \left(\frac{\left(\frac{1}{2n} \right) - 1}{-(1/2)} \right) = n^2 * -2 \left(\left(\frac{1}{2n} \right) - 1 \right) = n^2 * \left(-\frac{1}{n} + 2 \right) =$$

$$= n^2 * \left(2 - \frac{1}{n} \right) =$$

$$= 2n^2 - n$$

Árbol de recursión: Ejemplo 3 (usando la paradoja de Zenón)

Aplicando la serie geométrica:

$$n^2 \left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots + \frac{1^{\log_2 n}}{2} \right)$$

$$\text{Costo total} = n^2 * \left(\frac{r^k - 1}{r - 1} \right), r = \frac{1}{2}$$

$$= n^2 * \left(\frac{(1/2)^{\log_2 n + 1} - 1}{(1/2) - 1} \right)$$

$$= n^2 * \left(2 - \frac{1}{n} \right)$$

$$\text{Costo total } T(n) = 2n^2 - n$$

Complejidad: $O(n^2)$

B) O aproximamos, usando la paradoja de Zenón:

La suma infinita de $\left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots \right) = 2$ (nunca será > 2)

Pero **nuestro árbol no es infinito**, termina cuando el último

elemento es: $\frac{1^{\log_2 n}}{2}$

$$\text{Simplificando } \frac{1^{\log_2 n}}{2} = \frac{1^{\log_2 n}}{2^{\log_2 n}} = \frac{1}{n} \rightarrow 1^x = 1$$
$$\rightarrow \cancel{2^{\log_2 n}} = n$$

Árbol de recursión: Resumen de ejercicios

Ejercicio	Razón (r)	Costo total	Complejidad	¿Dónde está trabajo pesado?	¿Por qué?
1. $T(n) = 2T\left(\frac{n}{2}\right) + n$	$r = 1$	$n \log_2 n + n$	$O(n \log n)$	Equilibrado	El costo es igual en todos los niveles. El algoritmo es un reparto justo de tareas en toda la jerarquía.
2. $T(n) = 4T\left(\frac{n}{2}\right) + n$	$r = 2$	$2n^2 - n$	$O(n^2)$	En las Hojas	El costo se duplica en cada nivel. El algoritmo sufre por la cantidad de subproblemas.
3. $T(n) = 2T\left(\frac{n}{2}\right) + n^2$	$r = 1/2$	$2n^2 - n$	$O(n^2)$	En la Raíz	El costo se reduce a la mitad en cada nivel. El algoritmo sufre por la complejidad del paso inicial.



Teorema maestro

Teorema Maestro

- En análisis de algoritmos, el teorema maestro proporciona **límites asintóticos** para resolver de forma directa ecuaciones de recurrencia que ocurren en muchos algoritmos recursivos como en los de tipo ***Divide y Vencerás***.
- Fue popularizado por el libro *Introducción a los algoritmos (Cormen)*, en el cual fue tanto introducido como probado formalmente.
- No todas las ecuaciones de recurrencia pueden ser resueltas con el uso del teorema maestro.

Teorema Maestro

Nos da una “*receta*” para resolver ecuaciones de recurrencia que tienen la siguiente forma:

$$T(n) = aT\left(\frac{n}{b}\right) + \underbrace{f(n)}$$


Donde:

n : es el tamaño del problema a resolver

a : número de llamadas recursivas (número de subproblemas), es constante, $a \geq 1$

b : número de veces en que se divide el problema (tamaño del subproblema), es constante, $b > 1$

f(n): es el costo del trabajo realizado fuera de las llamadas recursivas (incluye el costo de la división del problema y el costo de unir las soluciones de los subproblemas). Es una función asintóticamente positiva.

Teorema maestro

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Subproblemas
(hojas)

Trabajo extra
(raíz)



Teorema maestro: 3 casos clave

En el último nivel del árbol hay $n^{\log_b a}$ subproblemas (nodos hoja). Esto significa que la recursión genera en total aproximadamente $n^{\log_b a}$ llamadas básicas. Por eso, se dice que **el costo total generado por la estructura de recursión** será:

$$n^{\log_b a}$$

Comparamos el crecimiento de $f(n)$ con el costo total de la recursión $n^{\log_b a}$:

$$\underline{f(n)} \quad VS \quad \underline{n^{\log_b a}}$$

Dependiendo de cuál domine, caeremos en uno de estos tres casos:

Caso 1: Los subproblemas dominan

Caso 2: Equilibrio

Caso 3: El trabajo extra domina

Teorema maestro: Caso 1

Caso 1:
**Los subproblemas
dominan**

Si:

$$\underline{f(n) < n^{\log_b a - \epsilon}, \epsilon > 0 \text{ (*)}}$$

Entonces:

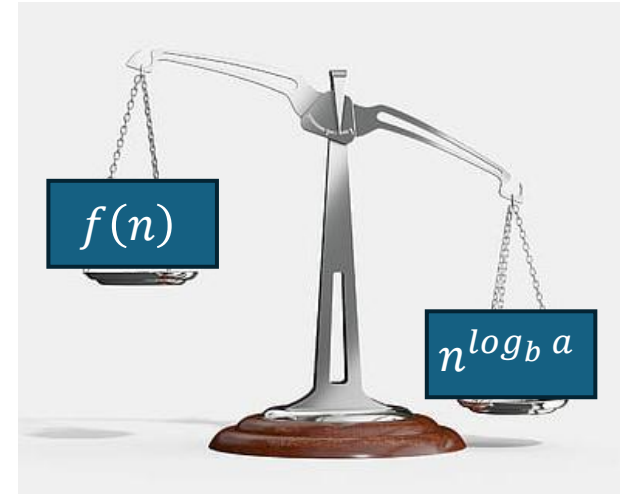
$$\underline{T(n) = \Theta(n^{\log_b a})}$$

$f(n)$ **crece mucho más lento**
que $n^{\log_b a}$

Significado: El trabajo pesado está ocurriendo en las **hojas** de la recursión.

(*) En este caso, $f(n)$ no solo es menor, sino que debe ser "polinomialmente menor" (debe haber una diferencia de n^ϵ).

Ejemplo: Si las hojas son n^2 y la raíz es n^1 , hay una brecha de n^1 (aquí $\epsilon = 1$).



Teorema maestro: Caso 2

Caso 2: Equilibrio

Si:

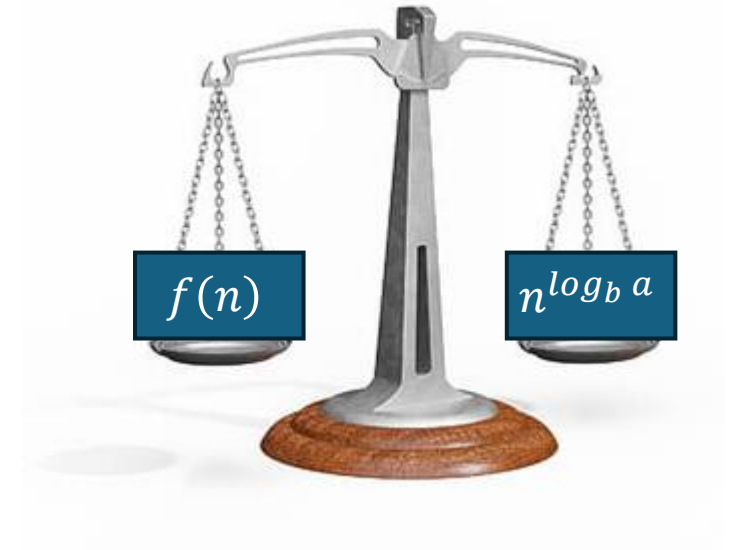
$$f(n) = n^{\log_b a}$$

Entonces:

$$\begin{aligned} T(n) &= \Theta(n^{\log_b a} \log n) \\ &= \Theta(f(n) \log n) \end{aligned}$$

Significado: El trabajo está **distribuido uniformemente** en todos los niveles del árbol.

$f(n)$ **crece igual** que $n^{\log_b a}$



Teorema maestro: Caso 3

Caso 3:
El trabajo extra
domina

Si:

$$f(n) > n^{\log_b a + \epsilon}, \epsilon > 0$$

y si:

$$af\left(\frac{n}{b}\right) \leq cf(n), (*)$$

$$c < 1 \text{ y}$$

n suficientemente grande

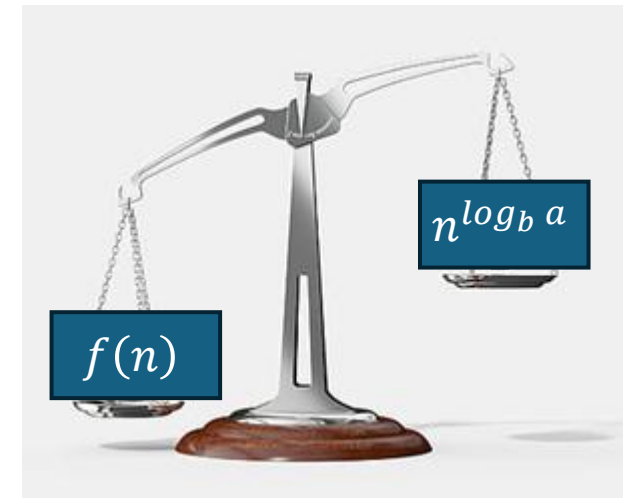
Entonces:

$$T(n) = \Theta(f(n))$$

Significado: El trabajo extra que realiza el nodo raíz (incluyendo dividir y combinar los problemas) es lo más costoso.

(*) Además de ser polinomialmente mayor, debe cumplir la **Condición de Regularidad**: el trabajo de los hijos debe ser una fracción constante del trabajo del padre.

$f(n)$ crece mucho más rápido
que $n^{\log_b a}$



Teorema maestro: Ejemplo 1

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

➤ Paso 1: Identificar valores

- $a = 2$
- $b = 2$
- $f(n) = n$

➤ Paso 2: Calcular $n^{\log_b a}$

- $n^{\log_b a} = n^{\log_2 2} \rightarrow \log_2 2 = 1$
 $= n^1$
 $= \underline{\underline{n}}$

Teorema maestro: Ejemplo 1

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

➤ **Paso 3: Comparar $f(n)$ vs $n^{\log_b a}$**

- $f(n) = n$
- $n^{\log_b a} = n$

Entonces:

$$f(n) = n^{\log_b a}$$

Por lo tanto aplica el **Caso 2.**

$$T(n) = \Theta(n^{\log_b a} \log n)$$

➤ **Paso 4: Resultado**

Sustituyendo $n^{\log_b a}$:

$$T(n) = \Theta(n \log n)$$

Complejidad = $O(n \log n)$

Teorema maestro: Ejemplo 2

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

Paso 1: Identificar valores

- $a = 4$
- $b = 2$
- $f(n) = n$

Paso 2: Calcular $n^{\log_b a}$

- $n^{\log_b a} = n^{\log_2 4} \rightarrow \log_2 4 = 2$
 $= \underline{\underline{n^2}}$

Teorema maestro: Ejemplo 2

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

Paso 3: Comparar $f(n)$ vs $n^{\log_b a}$

- $f(n) = n$
- $n^{\log_b a} = n^2$

Entonces:

$$f(n) < n^{\log_b a}$$

Por lo tanto aplica el **Caso 1.**

$$T(n) = \Theta(n^{\log_b a})$$

Paso 4: Resultado

Sustituyendo $n^{\log_b a}$:

$$T(n) = \Theta(n^2)$$

Complejidad = $O(n^2)$

Teorema maestro: Ejemplo 3

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

Paso 1: Identificar valores

- $a = 2$
- $b = 2$
- $f(n) = n^2$

Paso 2: Calcular $n^{\log_b a}$

- $n^{\log_b a} = n^{\log_2 2} \rightarrow \log_2 2 = 1$
 $= n^1$
 $= \underline{\underline{n}}$

Teorema maestro: Ejemplo 3

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

Paso 3: Comparar $f(n)$ vs $n^{\log_b a}$

- $f(n) = n^2$
- $n^{\log_b a} = n$

Entonces:

$$f(n) > n^{\log_b a}$$

Por lo tanto aplica el **Caso 3.**

$$T(n) = \Theta(f(n))$$

Paso 4: Resultado

Sustituyendo $f(n)$:

$$T(n) = \Theta(n^2)$$

Complejidad = $O(n^2)$

Teorema maestro: Resumen

Caso	Comparación	Qué significa	Complejidad
Caso 1	$f(n) < n^{\log_b a}$	La recursión domina	$T(n) = \Theta(n^{\log_b a})$
Caso 2	$f(n) = n^{\log_b a}$	Recursión y trabajo extra son iguales	$T(n) = \Theta(n^{\log_b a} \log n)$
Caso 3	$f(n) > n^{\log_b a}$	El trabajo extra domina	$T(n) = \Theta(f(n))$

Teorema maestro: Resumen de ejercicios

Ejercicio	a	b	$f(n)$	$n^{\log_b a}$	$f(n)$ vs $n^{\log_b a}$	Complejidad
1. $T(n) = 2T\left(\frac{n}{2}\right) + n$	2	2	n	n^2	$n < n^2$	$O(n^2)$
2. $T(n) = 4T\left(\frac{n}{2}\right) + n$	4	2	n	n	$n = n$	$O(n \log n)$
3. $T(n) = 2T\left(\frac{n}{2}\right) + n^2$	2	2	n^2	n	$n^2 > n$	$O(n^2)$

Teorema maestro: Casos irresolubles

- $T(n) = 2^n T\left(\frac{n}{2}\right) + n^2$ 

a no es una constante, el número de subproblemas debe ser fijo.

- $T(n) = 0.5T\left(\frac{n}{2}\right) + n$ 

$a < 1$, no puede darse el caso de que haya menos de un subproblema.

- $T(n) = 64T\left(\frac{n}{8}\right) - n^2$ 

$f(n)$ que es el tiempo de trabajo adicional, no puede ser negativo.



Continuamos

Búsqueda binaria recursiva

Merge sort

Quick sort

Bibliografía

- Lee, R.C.T., Tseng, S. S., Chang, R. C. y Tsai, Y. T. (2007). Introducción al diseño y análisis de algoritmos. Un enfoque estratégico. México, D.F. Mc Graw Hill.
- Cormen T. H., Leiserson C. E., R. L. Rivest y C. S. (2022). *Introduction to Algorithms* .
- <https://es.khanacademy.org/computing/computer-science/algorithms>
- <https://dipeshpatil.github.io/algorithms-visualiser/#/>
- <https://es.khanacademy.org/computing/computer-science/algorithms/recursive-algorithms/a/recursion>
- https://es.wikipedia.org/wiki/Teorema_maestro